LeetCode Solutions

LeetCode Assigned Problems

Name: Muhammad Farhan
Roll no: 25K-0538
Section: BCS-1K
Course: Programming Fundamentals Lab
Course Instructor: Sir Sheeraz Iqbal

1480. Running sum of 1d Array.

Code Accepted ×

All Submissions

Accepted 54 / 54 testcases passed

Farhan submitted at Nov 08, 2025 11:31

Editorial Solution

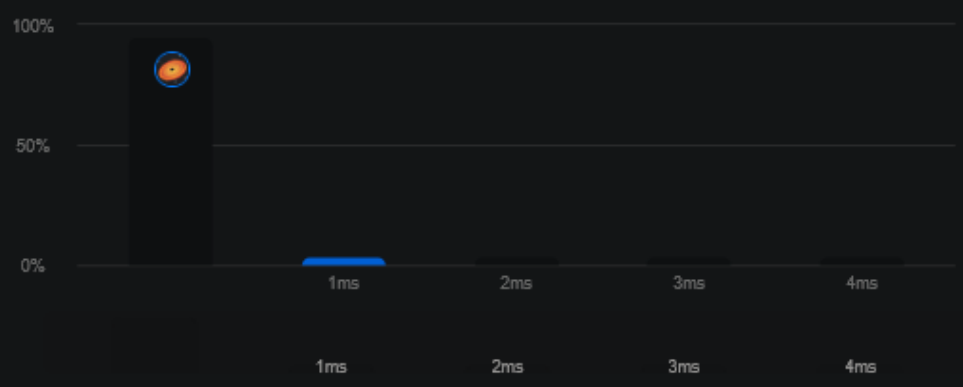
Runtime

0 ms Beats 100.00%

Analyze Complexity

Memory

11.20 MB Beats 44.23%



Code C

```c
/**
 * Note: The returned array must be malloced, assume caller calls free().
 */
 #include <stdlib.h>
int* runningSum(int* nums, int numsSize, int* returnSize) {
    *returnSize = numsSize;
    int* result = (int*)malloc(numsSize * 4);
    result[0]=nums[0];
    for (int i=1; i < numsSize ; i++)
    {
        result[i] = result[i-1]+nums[i];
    }
    return result;
}
```

View less

1470. Shuffle the Array.

Code Accepted ×

All Submissions

Accepted 53 / 53 testcases passed

Farhan submitted at Nov 08, 2025 15:57

Editorial Solution

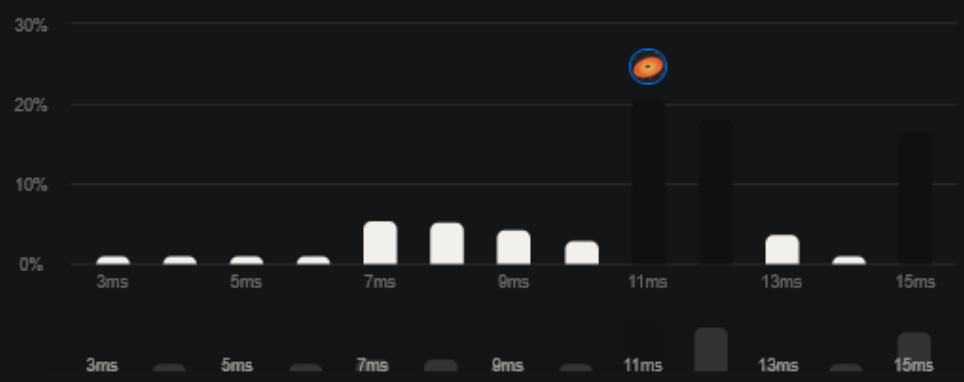
Runtime

11 ms Beats 80.11%

Analyze Complexity

Memory

11.50 MB Beats 87.84%



Code C

```c
/**
 * Note: The returned array must be malloced, assume caller calls free().
 */
 #include <stdlib.h>
int* shuffle(int* nums, int numsSize, int n, int* returnSize){
 *returnSize = numsSize;
int* result = (int*)malloc(numsSize * 4);
n = numsSize/2;
int o = n, e = 0;
for (int i=0; i < numsSize; i++)
{
    if (i%2 == 0)
    {
        result[i]=nums[e];
```

```c
/**
 * Note: The returned array must be malloced, assume caller calls free().
 */
 #include <stdlib.h>
int* shuffle(int* nums, int numsSize, int n, int* returnSize){
 *returnSize = numsSize;
int* result = (int*)malloc(numsSize * 4);
n = numsSize/2;
int o = n, e = 0;
for (int i=0; i < numsSize; i++)
{
    if (i%2 == 0)
    {
        result[i]=nums[e];
        e++;
    }
    else
    {
        result[i]=nums[o];
        o++;
    }
}
return result;
}
```

View less

3467. Transform Array by Parity

Code Accepted ×

All Submissions

Accepted 804 / 804 testcases passed

Farhan submitted at Nov 08, 2025 16:57

Solution

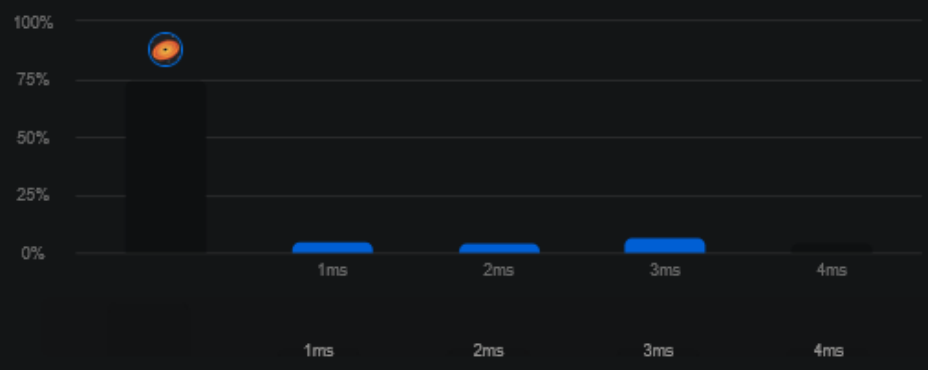
Runtime

0 ms Beats 100.00%

Analyze Complexity

Memory

11.76 MB Beats 91.24%



Code C

```c
/**
 * Note: The returned array must be malloced, assume caller calls free().
 */
int* transformArray(int* nums, int numsSize, int* returnSize) {
    *returnSize = numsSize;
    int* result = (int*)calloc(numsSize, 4);
    int k=0;
    for(int i=0; i<numsSize; i++)
    {
        if (nums[i]%2==1)
        k++;
    }
    for(int i=1; i<=k; i++)
    {
       result[numsSize-i]=1;
    }
return result;
```

26. Remove Duplicates from Sorted Array.

Initial and first Attempt:

Code Accepted ×

All Submissions

Accepted 362 / 362 testcases passed

Farhan submitted at Nov 13, 2025 20:24

Editorial Solution

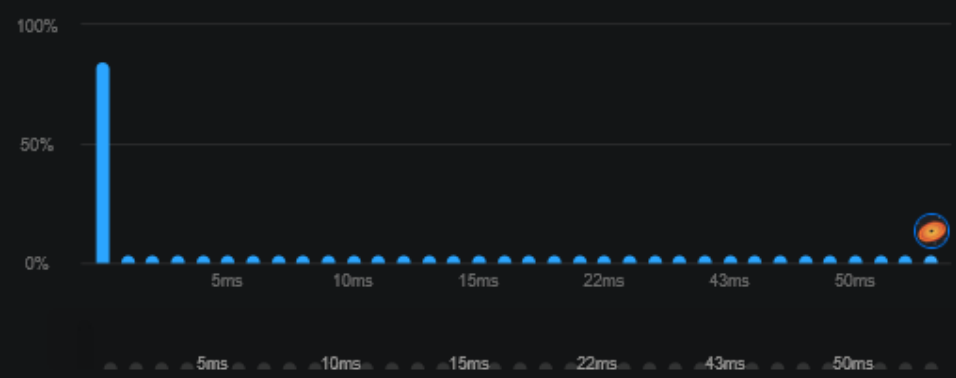
Runtime

782 ms Beats 5.01%

Analyze Complexity

Memory

12.47 MB Beats 12.79%



Code C

```c
int removeDuplicates(int* nums, int numsSize) {
    int temp[numsSize];
    int flag=0, k=0;
    for(int i=0; i<numsSize ; i++)
    {
        flag=0;
        for (int j=0; j<i ; j++)
        {
            if (nums[i]==nums[j])
            flag=1;
        }
        if (flag==0)
        {
            k++;
            temp[k-1]=nums[i];
        }
```

Code C

```c
int removeDuplicates(int* nums, int numsSize) {
    int temp[numsSize];
    int flag=0, k=0;
    for(int i=0; i<numsSize ; i++)
    {
        flag=0;
        for (int j=0; j<i ; j++)
        {
            if (nums[i]==nums[j])
            flag=1;
        }
        if (flag==0)
        {
            k++;
            temp[k-1]=nums[i];
        }
    }
    for (int i=0; i<k ; i++)
        nums[i]=temp[i];
return k;
}
```

View less

Second Attempt:

Code Accepted ×

All Submissions

Accepted 362 / 362 testcases passed

Farhan submitted at Nov 13, 2025 20:41

Editorial Solution

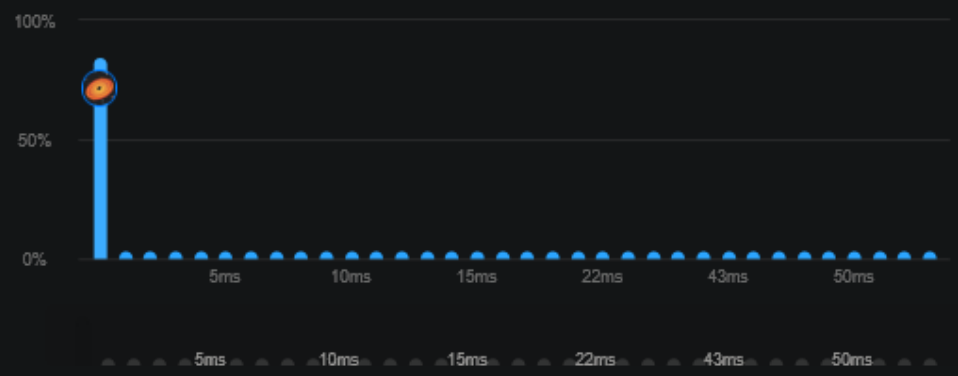
Runtime

0 ms Beats 100.00%

Analyze Complexity

Memory

12.25 MB Beats 27.17%



Code C

```c
int removeDuplicates(int* nums, int numsSize) {
   int k=1;
    for(int i=1; i<numsSize ; i++)
    {
        if (nums[i-1]!=nums[i])
        {
            nums[k]=nums[i];
            k++;
        }
    }
return k;
}
```

View less

344. Reverse String

Code Accepted ×

All Submissions

Accepted 477 / 477 testcases passed

Farhan submitted at Nov 13, 2025 19:46

Editorial Solution

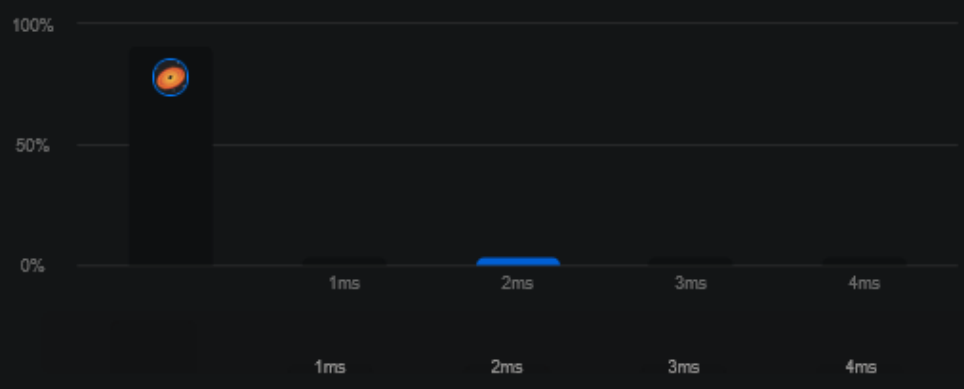
Runtime

0 ms Beats 100.00%

Analyze Complexity

Memory

17.16 MB Beats 41.08%



Code C

```c
void reverseString(char* s, int sSize) {
    char temp;
    int j;
    for(int i=0; i<sSize/2; i++)
    {
        j=sSize-1;
        j-=i;
        temp= s[i];
        s[i]=s[j];
        s[j]=temp;
    }
}
```

View less

283. Move Zeroes.

Code Accepted ×

All Submissions

Accepted 75 / 75 testcases passed

Farhan submitted at Nov 12, 2025 22:14

Editorial Solution

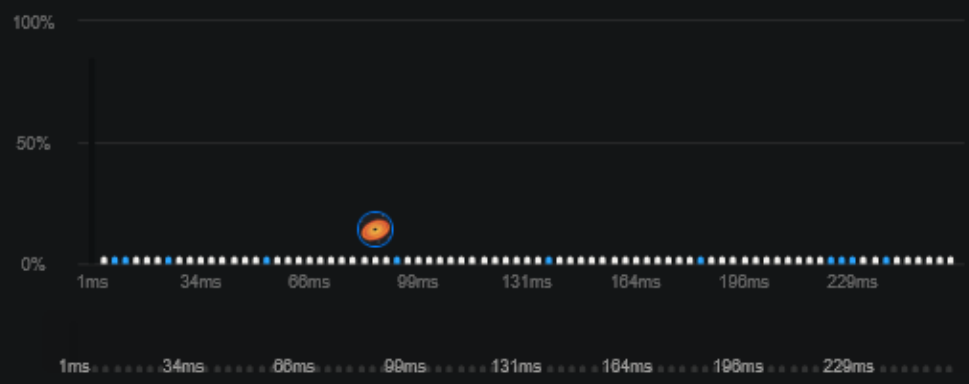
Runtime

88 ms Beats 10.34%

Analyze Complexity

Memory

19.70 MB Beats 46.31%



Code C

```c
void moveZeroes(int* nums, int numsSize) {
    int temp;
    for (int i=0; i<numsSize; i++)
    {
        if (nums[i]==0)
        {
            for (int j=i; j<numsSize; j++)
            {
                if (nums[j]!=0)
                {
                    temp=nums[j];
                    nums[j]=0;
                    nums[i]=temp;
                    break;
                }
            }
```

Code C

```c
void moveZeroes(int* nums, int numsSize) {
    int temp;
    for (int i=0; i<numsSize; i++)
    {
        if (nums[i]==0)
        {
            for (int j=i; j<numsSize; j++)
            {
                if (nums[j]!=0)
                {
                    temp=nums[j];
                    nums[j]=0;
                    nums[i]=temp;
                    break;
                }
            }
        }
    }
}
```

View less